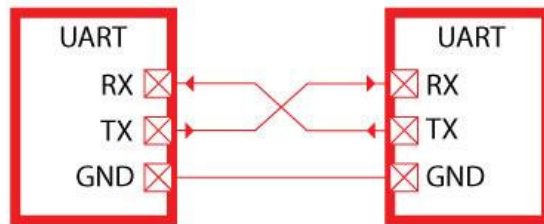


Lecture 10: ROM, RAM, and BRAM

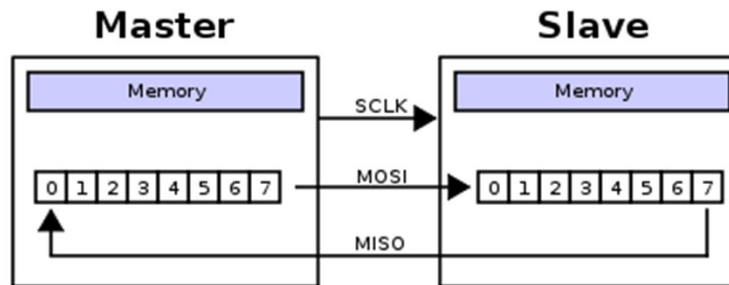
Emad Samuel Malki Ebeid

Summary of lecture 09

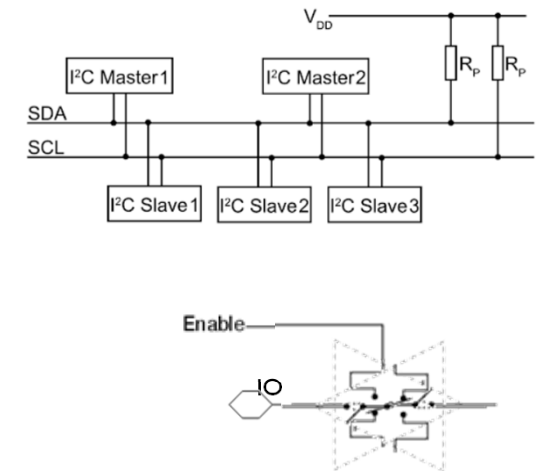
UART



SPI



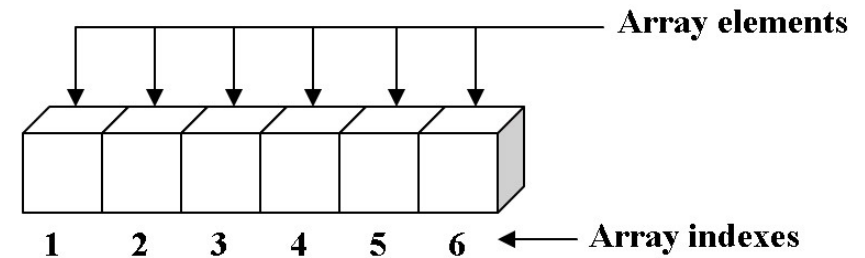
I2C



ROM

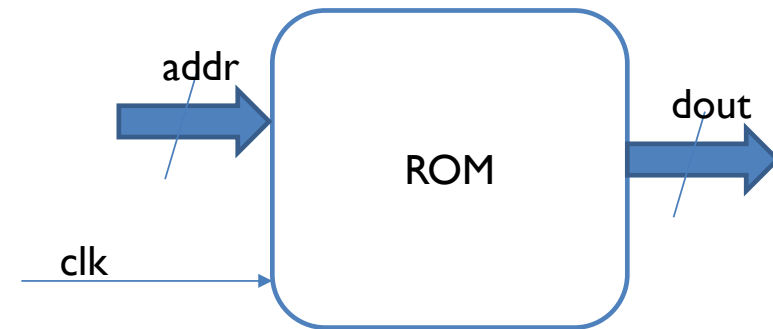
Array in VHDL

- Array has an index whose value selects each element.
- The index range determines:
 - Number of elements are in the array and their ordering (low to high, or high *downto* low).
- An index can be of any **integer** type.
- Array elements can be of **any** type.



One-dimensional array with six elements

ROM

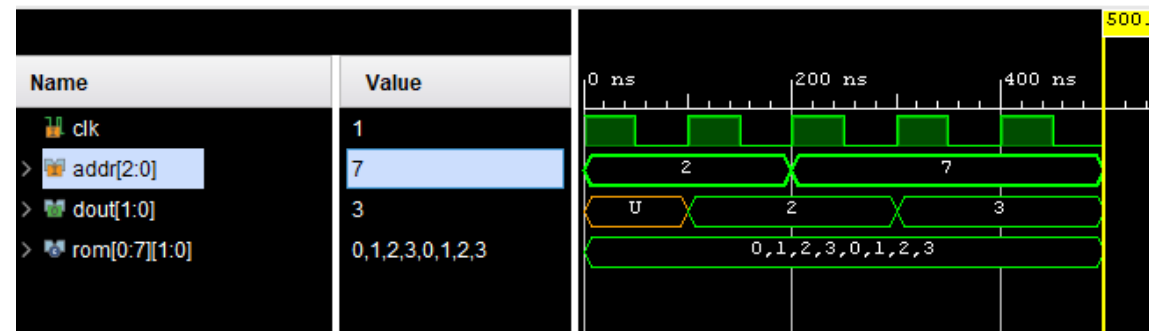


8x2

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_arith.all;

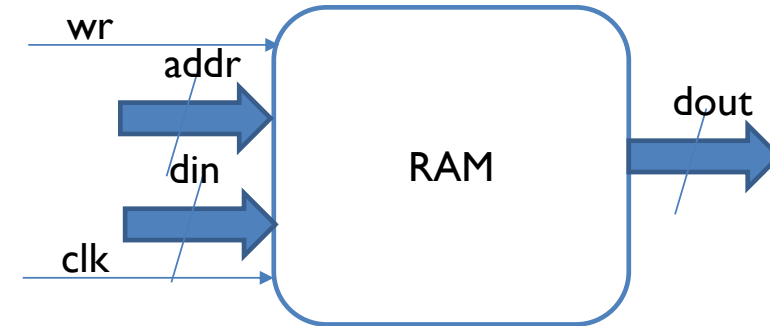
entity ROM_8_2 is
    Port ( clk : in STD_LOGIC;
          addr : in std_logic_vector(2 downto 0);
          dout : out std_logic_vector(1 downto 0)
    );
end ROM_8_2;

architecture Behavioral of ROM_8_2 is
    type rom_type is array (0 to 7) of std_logic_vector(1 downto 0);
    constant rom : rom_type:=("00","01","10","11","00","01","10","11");
BEGIN
    process(clk)
    begin
        if(rising_edge(clk))then
            dout<=rom(conv_integer(unsigned(addr)));
        end if;
    end process;
end Behavioral;
```



RAM

RAM



16x4

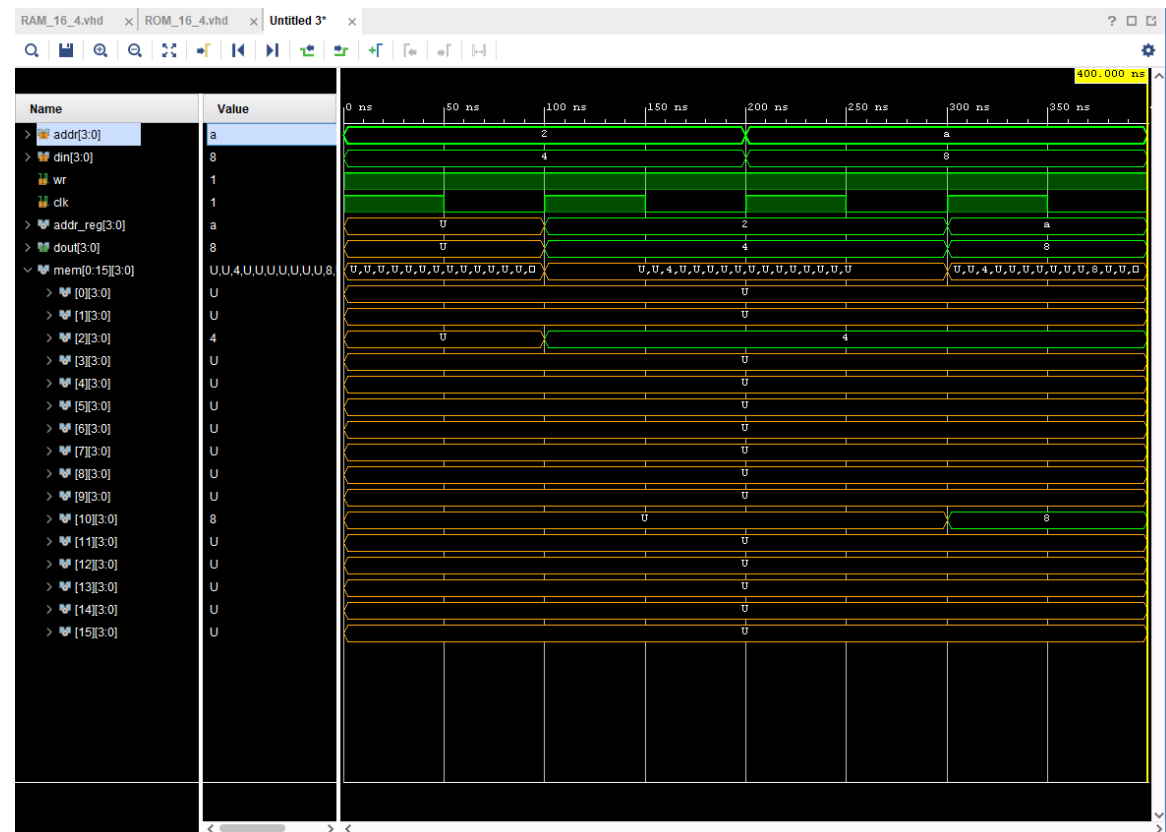
```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_arith.all;

entity RAM_16_4 is
    port(
        addr : in std_logic_vector(3 downto 0);
        din : in std_logic_vector(3 downto 0);
        wr,clk: in std_logic;
        dout :out std_logic_vector(3 downto 0)
    );
end RAM_16_4;

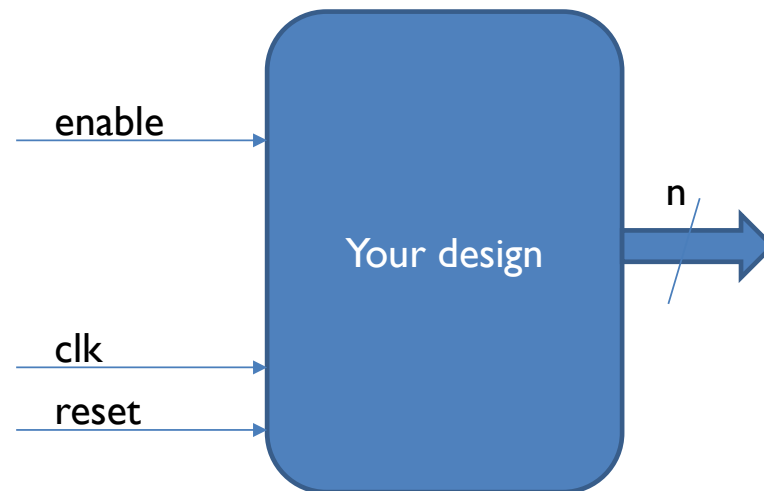
architecture Behavioral of RAM_16_4 is
    type mem_type is array (0 to 15) of std_logic_vector(3 downto 0);
    signal mem : mem_type;
    signal addr_reg:std_logic_vector(3 downto 0);
    BEGIN
        process(clk)
        begin
            if(rising_edge(clk))then
                if(wr='1')then
                    mem(conv_integer(unsigned(addr)))<=din;
                end if;
                addr_reg<=addr;
            end if;
        end process;
        dout<=mem(conv_integer(unsigned(addr_reg)));

    end Behavioral;
    
```



Excercise: Group work (20 minutes)

- Design a system that displays all ROM data when enabled.
 - ROM data is the group number.
 - For example: Group I
- Which is the size of ROM (address and 'n')?



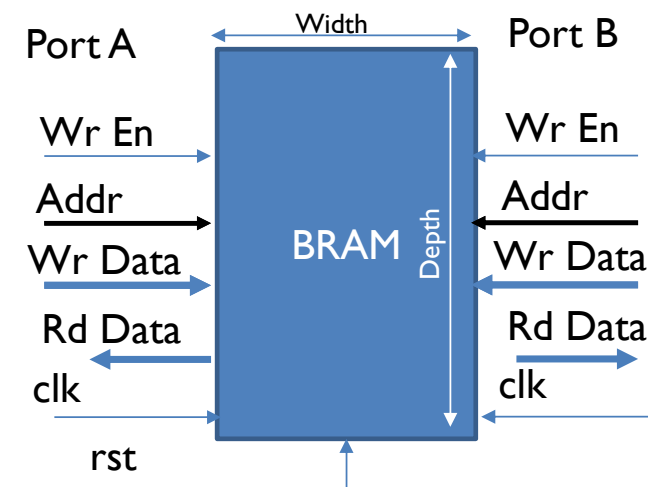
Block RAM

- Storing large amount of data (e.g., sensor data).
- Storing Read-only Data (calibration parameters).
- BRAM is a discrete parts of the FPGA.
- Each FPGA chip has a limited number of BRAM (Size=width*depth).
- Synchronous piece of logic
- Different configuration:
 - Single port, Dual port, FIFO

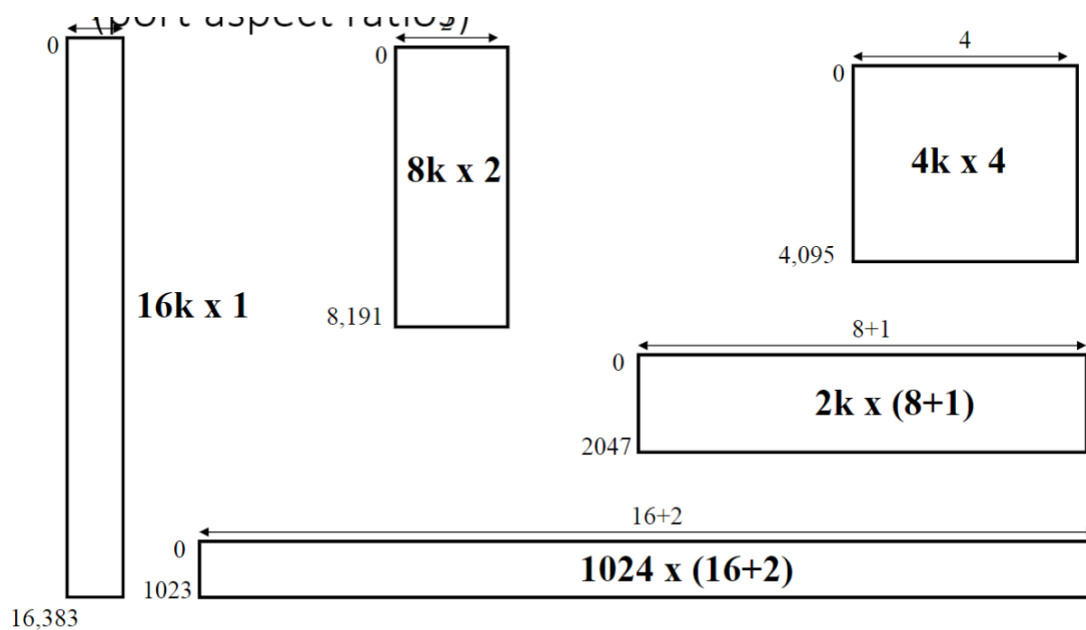
Wr En: Write Enable (1: write, 0: read)

Wr Data: Write data

Rd Data: Read Data



Block RAM configurations



Block RAM

Some of the key features of the block RAM include:

- Dual-port 36 Kb block RAM with port widths of up to 72
- Programmable FIFO logic
- Built-in optional error correction circuitry

Each device in the Zynq-7000 family has up to 755 dual-port block RAMs, each storing 36 Kb. Each block RAM has two completely independent ports that share nothing but the stored data.

Synchronous Operation

Each memory access, read or write, is controlled by the clock. All inputs, data, address, clock enables, and write enables are registered. The input address is always clocked, retaining data until the next operation. An optional output data pipeline register allows higher clock rates at the cost of an extra cycle of latency.

During a write operation, the data output can reflect either the previously stored data, the newly written data, or can remain unchanged.

Programmable Data Width

Each port can be configured as 32K x 1, 16K x 2, 8K x 4, 4K x 9 (or 8), 2K x 18 (or 16), 1K x 36 (or 32), or 512 x 72 (or 64). The two ports can have different aspect ratios without any constraints.

Each block RAM can be divided into two completely independent 18 Kb block RAMs that can each be configured to any aspect ratio from 16K x 1 to 512 x 36. Everything described previously for the full 36 Kb block RAM also applies to each of the smaller 18 Kb block RAMs.

Only in simple dual-port (SDP) mode can data widths of greater than 18 bits (18 Kb RAM) or 36 bits (36 Kb RAM) be accessed. In this mode, one port is dedicated to read operation, the other to write operation. In SDP mode, one side (read or write) can be variable, while the other is fixed to 32/36 or 64/72.

Both sides of the dual-port 36 Kb RAM can be of variable width.

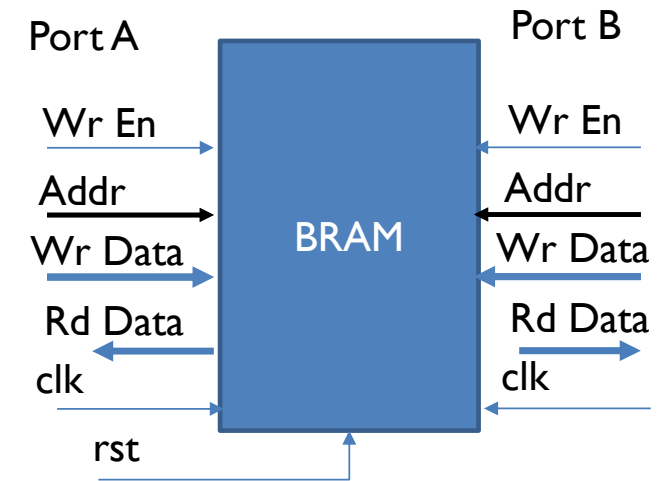
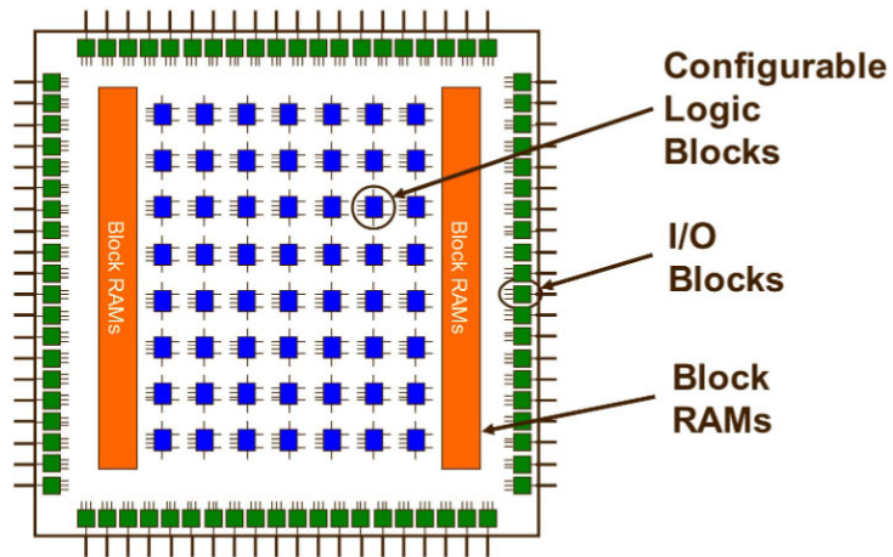
Two adjacent 36 Kb block RAMs can be configured as one cascaded 64K x 1 dual-port RAM without any additional logic.

Error Detection and Correction

Each 64-bit-wide block RAM can generate, store, and utilize eight additional Hamming code bits and perform single-bit error correction and double-bit error detection (ECC) during the read process. The ECC logic can also be used when writing to or reading from external 64- to 72-bit-wide memories.

Dual port BRAM configuration

- Each port can run in different clock frequency
- One operation at a time (read or write at the same address)!



Assignment 3

